AI-Assisted Development with C#

Course Outline

About This Course

The .NET ecosystem gives AI coding tools an unusually strong foundation to work against: a rich type system, Roslyn analyzers, and a mature test ecosystem provide fast, reliable feedback that agentic workflows thrive on. This five-day intensive teaches professional C# developers how to take full advantage, starting with how large language models generate code, setting up and configuring whichever tools your organization chooses in Visual Studio 2026 and VS Code, and prompting and context engineering techniques that produce idiomatic, nullable-aware, modern C# aligned with current .NET APIs (.NET 10 LTS today, with .NET 11 and C# 15 arriving in November 2026). Daily practice follows: disciplined inline assistance, chat-driven explain-refactor-generate workflows, grounding assistants in solution context with CLAUDE.md, AGENTS.md, and instructions files, and full agentic workflows where the coding agent your organization has chosen builds, reads compiler and analyzer output, and runs tests. Day four applies AI to quality across the lifecycle: xUnit suites, integration testing ASP.NET Core, AI-assisted debugging, refactoring at scale, modernizing legacy .NET Framework code, and documentation. Day five takes AI beyond the editor and into the delivery lifecycle in Azure DevOps (Boards, Repos, Pipelines, and GitHub Advanced Security) and then addresses team-scale adoption: NuGet supply-chain risks, secret handling, licensing and IP questions, organizational policy, cost management, and rolling AI out with honest productivity measurement.

- **Duration:** 35 hours of live training delivered over 5 days.
- **Delivery:** Instructor-led, with walkthroughs of prepared demos, hands-on activities, and reference documentation. All students receive a downloadable MP4 recording of the training.

- **Audience:** Professional C# developers who want a structured, end-to-end path from AI-curious to AI-proficient. Ideal for engineers and teams adopting AI coding tools across the full development lifecycle (writing, refactoring, testing, reviewing, and shipping .NET code) who also need to understand the security, licensing, and governance implications of doing so responsibly. This is a tool-agnostic course: each delivery is taught with the AI tools your organization chooses (GitHub Copilot, Claude Code, or another tool of your choosing), and the skills transfer to whichever assistants your team adopts. Labs run in both Visual Studio 2026 and VS Code, with delivery-lifecycle labs in Azure DevOps.
- **Prerequisites:**
  - Professional C# and .NET development experience
  - Working knowledge of Git and GitHub (issues, pull requests, branches, reviews)
  - Comfort with the command line and the .NET CLI
  - No prior AI tool experience required
- **Platform:** .NET 10 SDK (and the .NET 11 SDK once it ships), Visual Studio 2026 or VS Code with the C# Dev Kit, access to your organization's chosen AI coding tools (for example, GitHub Copilot or Claude Code; trials and free tiers are acceptable where the vendor offers them), Git, and the GitHub CLI.

What You Will Learn

- Explain how LLMs generate code, choose the right frontier model for a given C# task, and set up and configure your chosen AI tools in Visual Studio 2026 and VS Code with sound privacy and data controls.
- Apply prompting and context engineering techniques that produce idiomatic, nullable-aware, async-correct, modern C#.
- Use inline completions, next edit suggestions, and chat-driven explain, refactor, and generate workflows with discipline: well-scoped context, clean conversation hygiene, and review reflexes.
- Ground assistants and agents in solution context with CLAUDE.md, AGENTS.md, instructions files, skills, MCP servers, and shared team prompt libraries, and use AI to read and understand unfamiliar .NET solutions.
- Drive agentic coding sessions with your chosen coding agent, with dotnet build, analyzers, and tests in the loop, including issue-to-PR automation.
- Generate, evaluate, and maintain xUnit and integration test suites with AI assistance, and turn exceptions, async stack traces, and production logs into root causes faster.

- Refactor at scale and modernize legacy .NET Framework code to modern .NET behind characterization tests, and generate documentation: XML doc comments, READMEs, and architecture decision records.
- Put AI to work across the delivery lifecycle in Azure DevOps: connect agents to Boards, Repos, and Pipelines through the Azure DevOps MCP server, enable Copilot Code Review on Azure Repos, run agents inside pipelines, and use GitHub Advanced Security as a quality gate, with Copilot Autofix (limited preview) or an agent-drafted fix for its alerts.
- Recognize C#-specific AI failure modes (stale APIs, async anti-patterns, nullable violations), apply NuGet supply-chain and secret-handling guardrails, and contribute to team standards for cost, licensing, governance, rollout, and measuring real productivity impact.

Section I: Foundations: How AI Writes Code

- A. How LLMs generate code: tokens, context windows, and training data
- B. The frontier models for coding: Claude, GPT, and Gemini, and what the differences mean in practice
- C. Capability tradeoffs: speed, cost, context size, and reasoning
- D. Where AI excels in .NET work and where it reliably fails
- E. Setting expectations: assistant, pair programmer, or junior engineer?

Section II: Setting Up and Configuring Your Chosen Tools

- A. Accounts, seats, and enterprise policy: who gets access to what
- B. Plans, credits, and usage limits: what a seat actually buys
- C. Integration with Visual Studio 2026, VS Code, and the .NET CLI
- D. Model selection and switching: defaults, tiers, and per-task choices
- E. Privacy and data controls: retention, training opt-outs, and telemetry
- F. .NET-aware configuration: SDK versions, solutions, and the C# Dev Kit
- G. Verifying your setup: a smoke-test workflow before daily use

Section III: Prompting and Context Engineering for C#

- A. Writing prompts that produce idiomatic, modern C# (C# 14 on .NET 10, C# 15 on .NET 11)
- B. Steering toward nullable reference types, records, pattern matching, and extension members
- C. Async correctness: avoiding generated `async void` and sync-over-async
- D. Context strategies: solutions, projects, and what to include from large codebases
- E. Avoiding stale .NET Framework idioms in generated code

Section IV: Inline Completions and Chat Assistance

- A. Completions as a dialogue: writing code that invites good suggestions
- B. Next edit suggestions: letting the assistant follow a refactor through the file
- C. Steering completions with comments, XML docs, signatures, generics, and nullable annotations
- D. Explain, refactor, and generate: the core chat workflows, with context scoped to what the question needs
- E. From chat to edit: applying, reviewing, and adjusting suggested changes, and when to start a fresh conversation
- F. Review reflexes: interrogating generated C# for nullability, async paths, and edge cases, and staying in command of code you sign

Section V: IDE Assistants in Daily C# Work

- A. Your chosen assistant in Visual Studio 2026 and VS Code: completions, chat, agent mode, custom agents, and skills
- B. Visual Studio 2026's built-in agentic tooling: the Debugger Agent, Profiler Agent, Adaptive Paste, and test generation for .NET
- C. Terminal and desktop coding agents alongside the IDE: editor extensions, desktop apps, and working next to Visual Studio
- D. Multi-file edits and chat-driven refactoring across projects

Section VI: Working with Project Context

- A. CLAUDE.md, AGENTS.md, and copilot-instructions.md in depth: what belongs, what doesn't
- B. Path-scoped instructions, custom agents, and skills: one convention, many formats
- C. Encoding repo conventions: style, architecture, and testing norms
- D. Team prompt libraries: capturing prompts that work and sharing them
- E. Solution context: project layout, `Directory.Build.props`, and EditorConfig
- F. Keeping context files current as the codebase evolves

Section VII: Agentic Coding with Your Chosen Tools

- A. The agentic loop: plan, act, observe, iterate
- B. Your chosen coding agent in depth: setup, modes, and day-to-day operation
- C. Plan mode and the supervision spectrum: permission prompts to autopilot
- D. The verify loop: agents reading `dotnet build`, analyzer, and test output
- E. Subagents, hooks, and skills: packaging repeatable workflows for the team
- F. MCP servers: connecting agents to internal tools and data

Section VIII: AI-Driven Testing in Depth

- A. Generating xUnit suites: fixtures, theories, and coverage gaps
- B. IDE test generation versus agent-written suites: when each fits
- C. Integration testing ASP.NET Core with `WebApplicationFactory` and AI assistance
- D. Mocking and test doubles: steering AI toward your team's approach
- E. Designing for testability: letting AI surface seams and dependencies
- F. Mutation-style review: testing the tests AI writes
- G. Maintaining suites over time: updating tests as code changes

Section IX: AI-Assisted Debugging in Depth

- A. Turning exceptions and async stack traces into hypotheses
- B. Agentic debugging in Visual Studio 2026: the Debugger Agent from issue to reproducer to fix

- C. Deadlocks, sync-over-async, and threading issues with AI help
- D. Log analysis and structured logging for better AI diagnosis
- E. Debugging across solutions: services, middleware, and configuration
- F. Knowing when the AI is guessing: validating proposed fixes

Section X: Refactoring at Scale and Legacy Modernization

- A. Multi-project refactors: planning, executing, and reviewing with agents
- B. Characterization tests as the safety net before any change
- C. Modernizing legacy .NET Framework code to modern .NET incrementally
- D. Upgrade assistants and agent-driven upgrades: what they automate and what they miss
- E. C#-specific refactors: nullable adoption, async adoption, records and pattern matching
- F. Incremental modernization: strangler patterns over big rewrites
- G. Keeping diffs reviewable: small steps over heroic changes

Section XI: Code Comprehension and Documentation

- A. Onboarding to unfamiliar .NET solutions with AI guides, explanations, call graphs, and architecture diagrams
- B. Tracing dependency injection, middleware, and async flows, and reconstructing intent from legacy code and history
- C. Summarizing projects, namespaces, and pull requests
- D. XML doc comments that match project conventions, and public API documentation with DocFX
- E. READMEs, how-to guides, onboarding docs, and architecture decision records from real code
- F. Keeping documentation honest: reviewing generated prose for drift

Section XII: C#-Specific Pitfalls and Verification

- A. Hallucinated NuGet packages and API versions
- B. Stale framework knowledge: verifying against current .NET docs when the model predates C# 14 and 15

- C. Nullable violations and analyzer warnings as quality gates
- D. EditorConfig, Roslyn analyzers, and `dotnet format` for generated code
- E. Performance review: allocations, LINQ overuse, and async overhead

Section XIII: AI Across the Delivery Lifecycle with Azure DevOps

- A. Where AI fits in the lifecycle: backlog refinement, implementation, review, build, release, and operations
- B. Connecting agents to Azure DevOps: the remote and local MCP servers for work items, pull requests, repos, pipelines, and wiki, and which clients each supports
- C. Azure Boards with AI: drafting and refining work items and acceptance criteria from the IDE, and keeping work items linked to AI-assisted branches and pull requests
- D. Copilot Code Review for Azure Repos (public preview): enabling it in organization, project, and repository settings, making it automatic with a branch policy, custom review instructions at organization, project, repository, and path level, and acting on the results
- E. Work item to pull request automation: what the Copilot cloud agent integration requires and the alternatives for code in Azure Repos
- F. Agents in Azure Pipelines: running coding agents non-interactively in YAML jobs for pull request summaries, failure triage, and release notes (an adapted pattern; there is no official guide yet)
- G. GitHub Advanced Security for Azure DevOps as a quality gate: CodeQL scanning, Copilot Autofix for alerts (limited preview, with an agent-drafted fix as the fallback), and dependency and secret scanning
- H. Cost visibility: AI credits, per-project attribution in Azure Cost Management for the charges that carry project tags, and budget alerts

Section XIV: Security, Licensing, and Governance

- A. Prompt injection and untrusted content in agent workflows
- B. Secret leakage: user secrets, appsettings, and context hygiene
- C. NuGet supply-chain vetting and lockfile discipline
- D. Licensing and IP status of AI-generated code
- E. Organizational policy: approved tools, data boundaries, and audit trails
- F. Review requirements: keeping humans accountable for shipped code

Section XV: Measuring Impact, Cost, and Team Adoption

- A. Re-evaluating the tool landscape: selection criteria, trials, and switching costs
- B. Rollout patterns that work: champions, guilds, and paired onboarding
- C. Metrics that matter, and the vanity metrics that mislead
- D. Adoption anti-patterns: mandates, leaderboards, and unreviewed merges